```
// ... $xml, $dom, and $xpath init as before
$results = $xpath->query(
    '//project[contains(@class, "php")] | //project[contains(@class,
                                        "book")]'
);
print "Found {$results->length} matching node(s)" . PHP_EOL;
foreach ($results as $node) {
    var_dump(trim($node->textContent));
}
```

... but do bear in mind performance issues for large DOMs or complex queries (the union requires two passes of the DOM, the predicate with an `or` requires one).

But I'm Not Running PHP 8.4 :(

We're pretty much done with looking at creating and navigating the DOM; the next logical step is making modifications. Before we do that, though, I promised to explain why our XPath examples were all using XML and not HTML. The reason for this is HTML5 and PHP's support for it. Prior to version 8.4, PHP didn't have a proper HTML5-compliant parser, which is a little mind-boggling considering what PHP is designed to do, not to mention the fact that HTML5 is knocking on for sixteen years old now... but here we are. Native HTML parsing before 8.4 was achieved with `DOMDocument` (which still exists in PHP 8.4), and the underlying XML implementation for `DOMDocument` (libxml2) only supports HTML up to version 4.01. One of the big differences between HTML versions 5 and <= 4.01 is the DOCTYPE declaration: HTML 4 and earlier DOCTYPE contained extra information called a DTD (Document Type Declaration), which is a schema document: a declaration of all the elements and their attributes that are valid in the document. Prior to HTML5, a DOCTYPE line would look something like this: `<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">`, where `http://www.w3.org/TR/html4/strict.dtd` is used as the schema (the rules that define how valid documents should be structured), you can feed that URL into a browser and have a look for yourself if you feel inclined!

HTML5 did away with dtd's in the DOCTYPE. The end result is that `DOMDocument` has no DTD to work with when parsing an HTML5 `<!DOCTYPE ...>` statement, so it falls back to a default (HTML 4), and this is what happens if `DOMDocument` is given some HTML5 elements to parse: